User Stories & Acceptance Criteria

PayTrack | Prioritised backlog examples using testable outcomes

SAMPLE ONLY — FICTIONAL PORTFOLIO CASE STUDY. All names, records, targets, dates and amounts are illustrative, not evidence of actual client work.

Document version 1.0 | Scenario: PayTrack payment reconciliation portal | Owner: Sample Business Analyst

1. How to use this sample backlog

Stories are grouped around a minimal pilot increment. Acceptance criteria use Given / When / Then to make behavior observable and support QA traceability. Estimates, sprint assignment and final priority would be agreed with the delivery team.

US-01 | As an Operations Analyst, I want a consolidated reconciliation overview so that I can understand the current workload.

Priority Must | Trace: BR-01 / FR-002

- Given a selected time window, when the overview loads, then summary counts reflect records in that window.
- Given the source job failed, when the view opens, then the latest run status and last successful timestamp are visible.
- Given the user can access only a subset of records, when metrics are displayed, then they respect the same access scope.

US-02 | As an Operations Analyst, I want to search a payment by reference so that I can investigate a specific customer report.

Priority Must | Trace: BR-01 / FR-003

- Given a valid payment ID or reference, when I search, then matching permitted records are returned.
- Given no match exists, when the search completes, then a clear empty state appears.
- Given a malformed query, when submitted, then the input is handled safely without exposing a stack trace.

US-03 | As an Operations Analyst, I want to compare internal and gateway fields so that I can understand the reconciliation outcome.

Priority Must | Trace: BR-02 / FR-004–006

- Given a transaction has both source values, when details are opened, then amount, currency, reference and status are shown side by side.
- Given a mismatch exists, then the reason code and compared values are visible.
- Given source data is missing, then the UI shows “Unavailable” rather than substituting a guessed value.

US-04 | As an Operations Analyst, I want exceptions created for unmatched records so that work is not lost in manual trackers.

Priority Must | Trace: BR-02 / FR-007

- Given an eligible unmatched transaction, when the reconciliation run completes, then an exception case exists with a reason code.

- Given the same event is replayed, then no duplicate open case is created for the configured uniqueness key.
- Given a match becomes valid after a new event, then the case follows the approved status transition rule.

US-05 | As an Operations Analyst, I want to assign and comment on a case so that work has a clear owner and investigation history.

Priority Must | Trace: BR-03 / FR-008–009

- Given I have assignment permission, when I assign an eligible owner, then case ownership updates and an audit event is recorded.
- Given I add a note, when it is saved, then the note appears with actor and timestamp.
- Given I lack permission, when I attempt either action, then the server rejects it and the record remains unchanged.

US-06 | As an authorised user, I want to resolve a case with a reason so that the outcome is clear and auditable.

Priority Must | Trace: BR-03 / FR-010–012

- Given required resolution type or reason is missing, when I submit, then field validation prevents submission.
- Given the role is authorised and policy does not require approval, when I submit a valid resolution, then the case status becomes Resolved and an audit record is added.
- Given approval is required, when I submit, then the case moves to Pending Approval and does not become Resolved until approval.

US-07 | As an Operations Lead, I want to review and approve escalated resolutions so that controlled actions follow policy.

Priority Should | Trace: BR-03 / FR-011

- Given a case requires approval, when an authorised lead opens it, then requested action and supporting reason are visible.
- Given approval is granted or rejected, when a decision is saved, then actor, timestamp and decision are recorded.
- Given the same decision is submitted twice, then duplicate transitions are prevented.

US-08 | As a Support user, I want read-only transaction status so that I can answer operational questions without changing records.

Priority Must | Trace: BR-05 / FR-001, FR-004

- Given a Read-only role, when transaction details are opened, then permitted fields are visible.
- Given a read-only user attempts to resolve, assign or edit, then the action is unavailable in the UI and denied by the server.
- Given sensitive fields are restricted, then they are omitted or masked according to policy.

US-09 | As an Operations Analyst, I want to filter the exception queue so that I can work the most urgent cases first.

Priority Should | Trace: BR-06 / FR-013

- Given status, reason, owner and date filters, when applied, then results satisfy all selected filters.
- Given a filter returns no cases, then the page presents an empty state and a way to clear filters.
- Given filters are changed, then count and displayed rows remain consistent.

US-10 | As a Platform Operator, I want ingestion and reconciliation job status so that I can detect source-feed issues.

Priority Should | Trace: BR-08 / FR-015

- Given a job completed, then status and last-run time are displayed.
- Given a job failed, then a safe error summary and next action are shown.
- Given a retry is initiated, then duplicate processing is prevented by idempotency controls.

2. Definition of Ready (sample)

- Business outcome and actor are clear.
- Acceptance criteria are testable and cover validation/permission edges.
- Dependencies, source fields and unresolved policy decisions are recorded.
- Required design or process reference is linked.
- Data privacy and audit implications are reviewed for the story.

3. Definition of Done (sample)

- Acceptance criteria pass in the agreed test environment.
- Unit/integration/regression evidence is recorded where applicable.
- Access-control and audit assertions are tested for relevant actions.
- User-facing error and empty states are verified.
- Requirements traceability and release notes are updated; no unresolved critical defect remains.